Locality Sensitive Hashing (LSH)

Baptiste Jeudy
baptiste.jeudy@univ-st-etienne.fr

Laboratoire Hubert Curien,
Université Jean-Monnet Saint-Etienne.

Master MLDM / DSC

Part I:
Locality Sensitive Hashing (LSH)

Problem and Applications

Given a large set of elements:

Finding groups of similar elements

- Web pages / documents (clustering, plagiarism detection)
- Cluster news pages (press agency)
- Finding near duplicates in a database (for correction / fusion)
  - Customer database
  - Bibliographic database

Finding Nearest Neighbors

- Classification (images, documents, ...)
- Recommendation (movies, web pages, products)
- Search

Example: Near Duplicates

- Bibliographic Database
- Objective: remove near duplicates
- Example of near duplicates
  1. The Art of Computer Programming, D. E. Knuth
  2. Art of Computer Programming (The), Donald E. Knuth
  3. The art of computer programming, D. Knuth
  4. The Art of Computer Programing, Donald Knuth
  5. ...

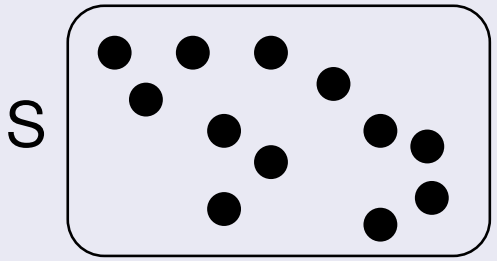
Problem Formalization

Given:

- a large set S (documents, images, web pages, customers,...)
- a similarity function Sim: $S^2 \rightarrow \mathbb{R}^+$

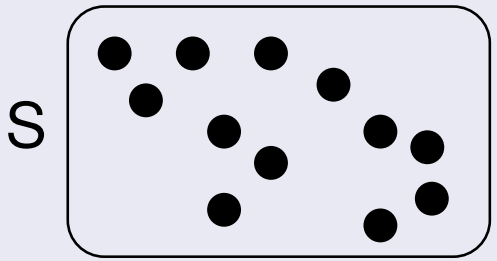
Finding groups of similar elements

- Find all pairs $(a, b) \in S^2$ of similar elements, i.e., $\text{Sim}(a, b) > \sigma$ where σ is a user defined threshold.



Nearest Neighbor (NN or 1-NN):

- Given a query element q
- Find the element $s \in S$ the most similar to q
  $s = \text{argmax}_{x \in S} \text{Sim}(q, x)$
- Variation: the k-NN (k Nearest Neighbors)



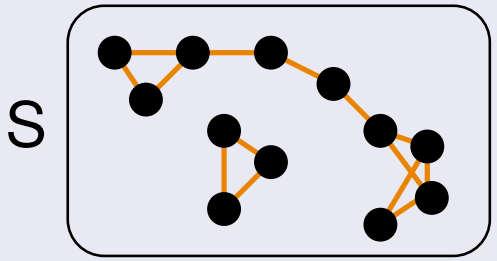
Problem Formalization

Given:

- a large set S (documents, images, web pages, customers,...)
- a similarity function Sim: $S^2 \rightarrow \mathbb{R}^+$

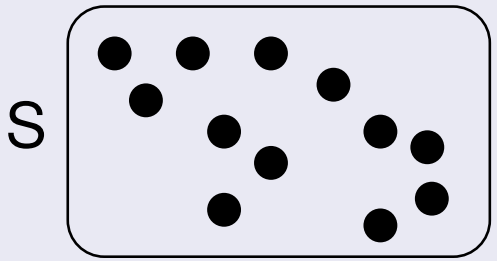
Finding groups of similar elements

- Find all pairs $(a, b) \in S^2$ of similar elements, i.e., $\text{Sim}(a, b) > \sigma$ where σ is a user defined threshold.



Nearest Neighbor (NN or 1-NN):

- Given a query element q
- Find the element $s \in S$ the most similar to q
  $s = \text{argmax}_{x \in S} \text{Sim}(q, x)$
- Variation: the k-NN (k Nearest Neighbors)



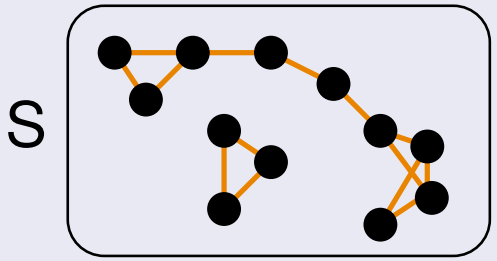
Problem Formalization

Given:

- a large set S (documents, images, web pages, customers,...)
- a similarity function Sim: $S^2 \rightarrow \mathbb{R}^+$

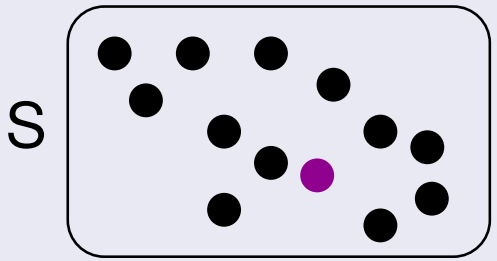
Finding groups of similar elements

- Find all pairs $(a, b) \in S^2$ of similar elements, i.e., $\text{Sim}(a, b) > \sigma$ where σ is a user defined threshold.



Nearest Neighbor (NN or 1-NN):

- Given a query element q
- Find the element $s \in S$ the most similar to q
  $s = \text{argmax}_{x \in S} \text{Sim}(q, x)$
- Variation: the k-NN (k Nearest Neighbors)



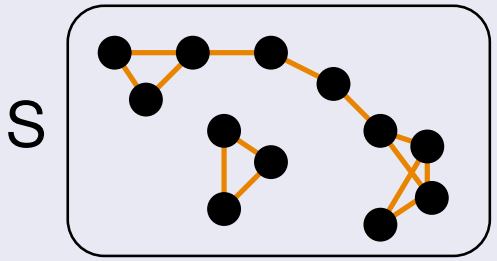
Problem Formalization

Given:

- a large set S (documents, images, web pages, customers,...)
- a similarity function Sim: $S^2 \rightarrow \mathbb{R}^+$

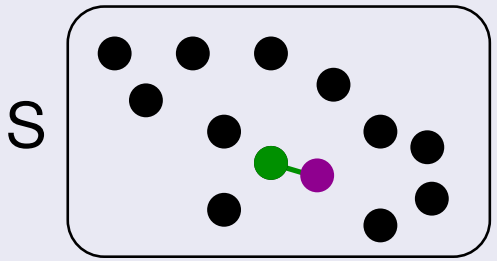
Finding groups of similar elements

- Find all pairs $(a, b) \in S^2$ of similar elements, i.e., $\text{Sim}(a, b) > \sigma$ where σ is a user defined threshold.



Nearest Neighbor (NN or 1-NN):

- Given a query element q
- Find the element $s \in S$ the most similar to q
  $s = \text{argmax}_{x \in S} \text{Sim}(q, x)$
- Variation: the k-NN (k Nearest Neighbors)



Similarity Functions

Similarity Function Sim: $S^2 \rightarrow [0, 1]$

- Sim(a, b) higher if a and b similar
  - Sim$(a, b) = 0$ if a and b very different
  - Sim$(a, b) = 1$ if $a = b$ or very similar.
- Can be defined with a distance function:
  Sim$(a, b) = 1/dist(a, b)$ or Sim$(a, b) = 1 - dist(a, b)$.
- Depends on the application (and kind of elements in S)

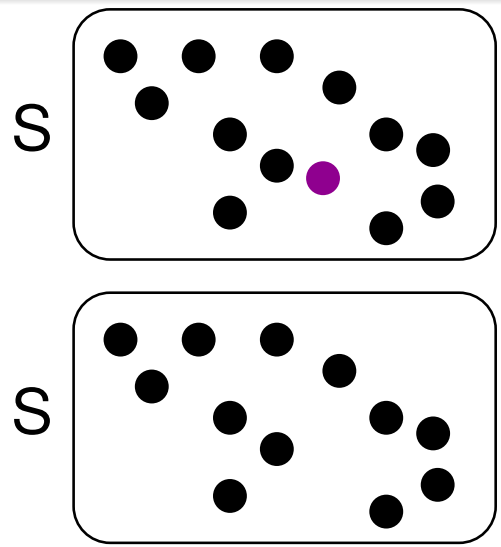
LSH Motivation: Complexity

Complexity

Number of similarity function computations ($\times$ the complexity of similarity computation)

S has n elements

- Nearest Neighbor: for one query q: $O(n)$
  - Need to compute $\text{Sim}(q, x)$ for all $x \in S$
- Finding groups of similar elements:
  - Compute similarity of all pairs $(a, b) \in S^2$
  - $n(n-1)/2 = O(n^2)$
- n can be in the order of millions / billions.
- Each similarity computation can be expensive (images, documents)
- On search engines: hundreds/thousands of queries per second.



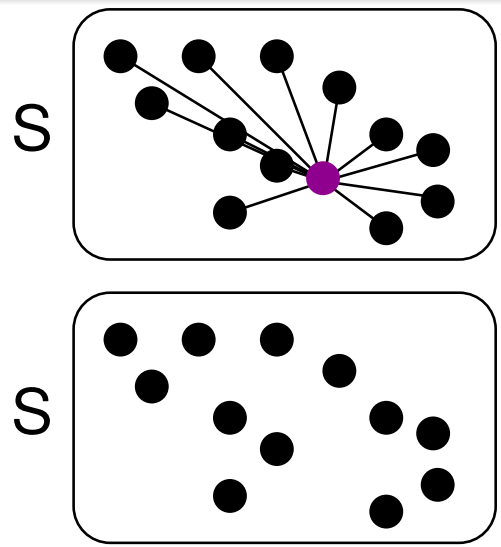
LSH Motivation: Complexity

Complexity

Number of similarity function computations ($\times$ the complexity of similarity computation)

S has n elements

- Nearest Neighbor: for one query q: $O(n)$
  - Need to compute $\text{Sim}(q, x)$ for all $x \in S$
- Finding groups of similar elements:
  - Compute similarity of all pairs $(a, b) \in S^2$
  - $n(n-1)/2 = O(n^2)$
- n can be in the order of millions / billions.
- Each similarity computation can be expensive (images, documents)
- On search engines: hundreds/thousands of queries per second.



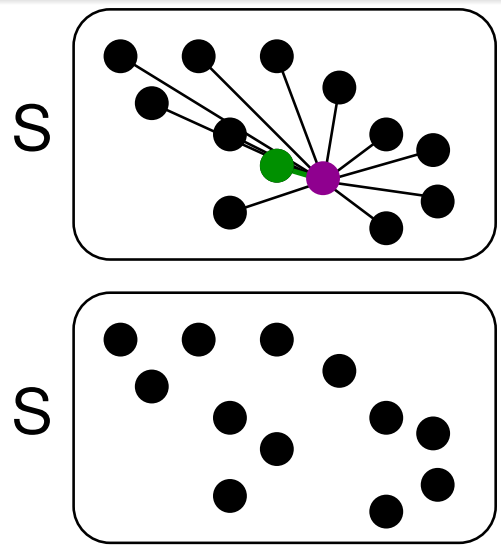
LSH Motivation: Complexity

Complexity

Number of similarity function computations ($\times$ the complexity of similarity computation)

S has n elements

- Nearest Neighbor: for one query q: $O(n)$
  - Need to compute $\text{Sim}(q, x)$ for all $x \in S$
- Finding groups of similar elements:
  - Compute similarity of all pairs $(a, b) \in S^2$
  - $n(n-1)/2 = O(n^2)$
- n can be in the order of millions / billions.
- Each similarity computation can be expensive (images, documents)
- On search engines: hundreds/thousands of queries per second.



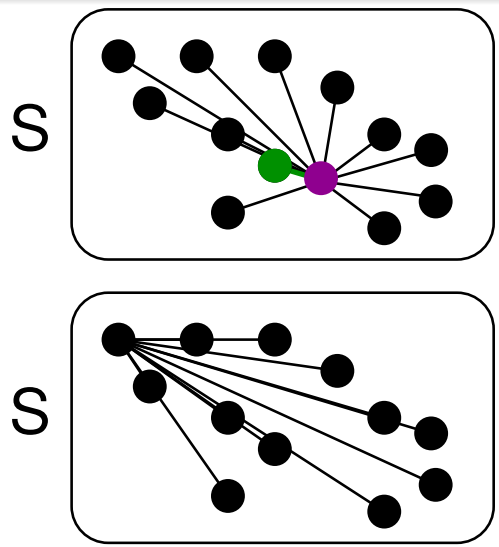
LSH Motivation: Complexity

Complexity

Number of similarity function computations ($\times$ the complexity of similarity computation)

S has n elements

- Nearest Neighbor: for one query q: $O(n)$
  - Need to compute $\text{Sim}(q, x)$ for all $x \in S$
- Finding groups of similar elements:
  - Compute similarity of all pairs $(a, b) \in S^2$
  - $n(n-1)/2 = O(n^2)$
- n can be in the order of millions / billions.
- Each similarity computation can be expensive (images, documents)
- On search engines: hundreds/thousands of queries per second.



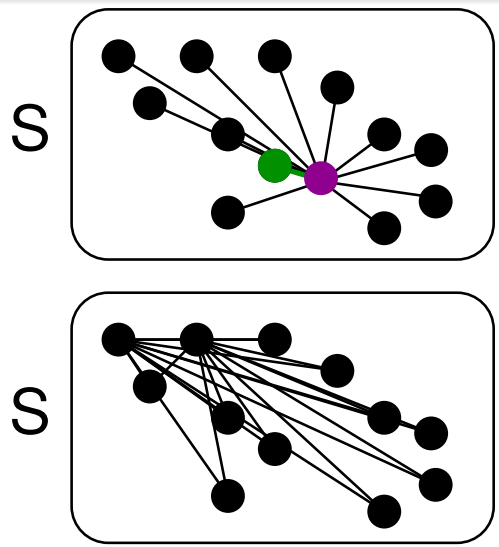
LSH Motivation: Complexity

Complexity

Number of similarity function computations ($\times$ the complexity of similarity computation)

S has n elements

- Nearest Neighbor: for one query q: $O(n)$
  - Need to compute $\text{Sim}(q, x)$ for all $x \in S$
- Finding groups of similar elements:
  - Compute similarity of all pairs $(a, b) \in S^2$
  - $n(n-1)/2 = O(n^2)$
- n can be in the order of millions / billions.
- Each similarity computation can be expensive (images, documents)
- On search engines: hundreds/thousands of queries per second.



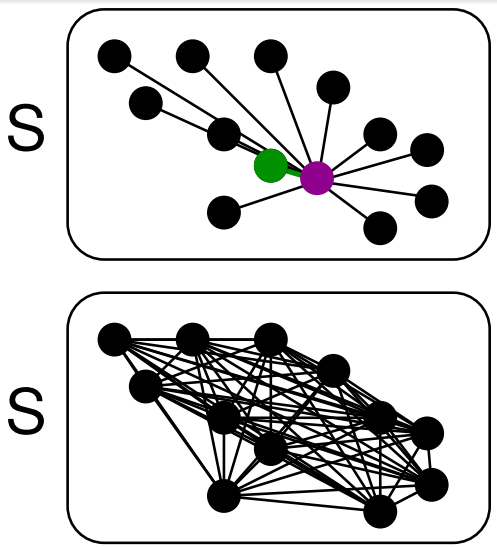
LSH Motivation: Complexity

Complexity

Number of similarity function computations ($\times$ the complexity of similarity computation)

S has n elements

- Nearest Neighbor: for one query q: $O(n)$
  - Need to compute $\text{Sim}(q, x)$ for all $x \in S$
- Finding groups of similar elements:
  - Compute similarity of all pairs $(a, b) \in S^2$
  - $n(n-1)/2 = O(n^2)$
- n can be in the order of millions / billions.
- Each similarity computation can be expensive (images, documents)
- On search engines: hundreds/thousands of queries per second.



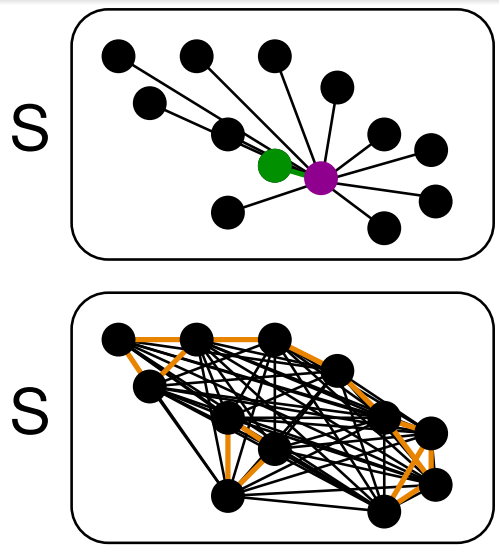
LSH Motivation: Complexity

Complexity

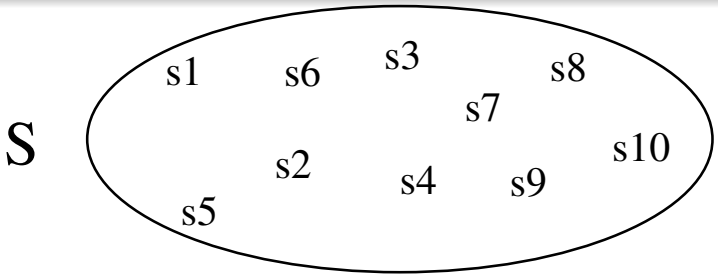
Number of similarity function computations ($\times$ the complexity of similarity computation)

S has n elements

- Nearest Neighbor: for one query q: $O(n)$
  - Need to compute $\text{Sim}(q, x)$ for all $x \in S$
- Finding groups of similar elements:
  - Compute similarity of all pairs $(a, b) \in S^2$
  - $n(n-1)/2 = O(n^2)$
- n can be in the order of millions / billions.
- Each similarity computation can be expensive (images, documents)
- On search engines: hundreds/thousands of queries per second.

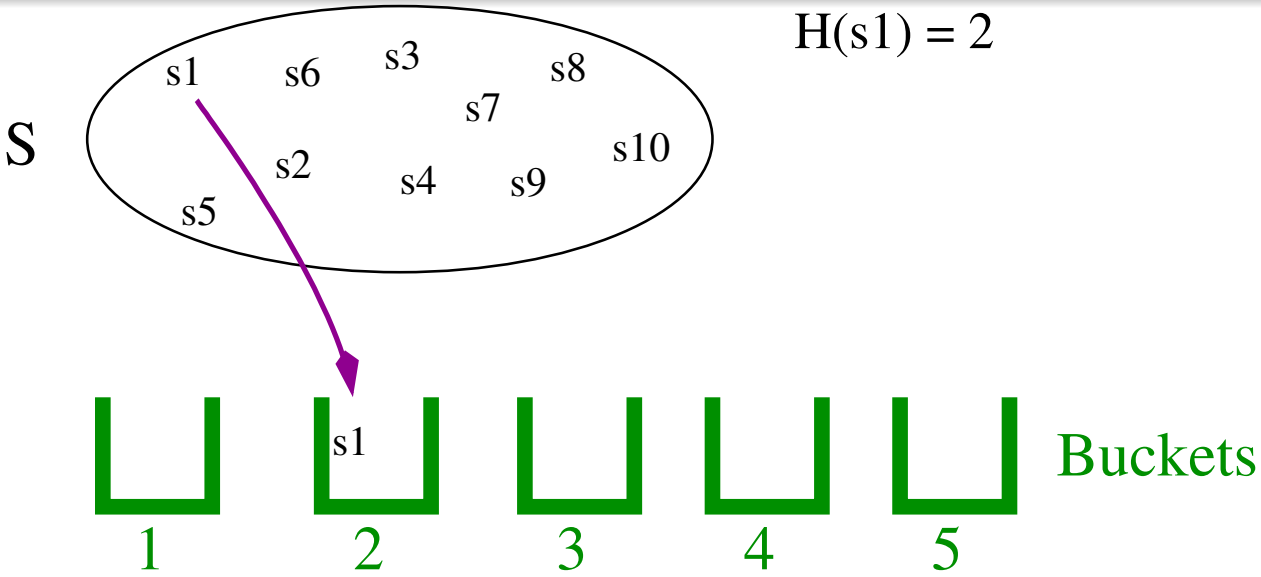


Locality Sensitive Hashing: Big Picture



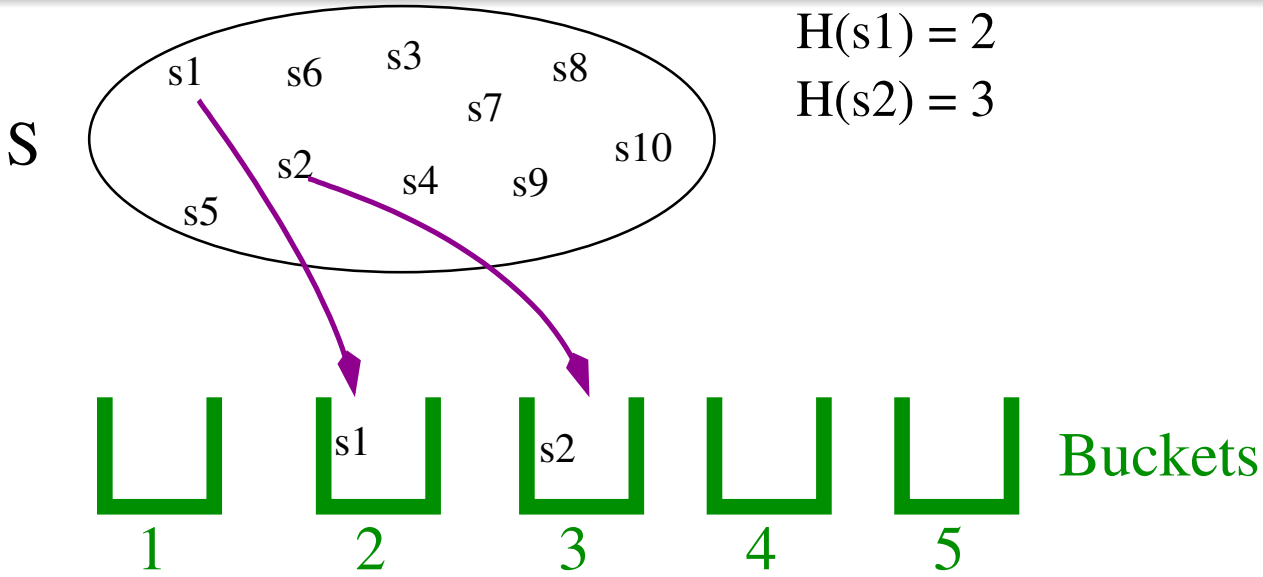
- Put elements of S into several "buckets" such that (with high probability):
  - Similar elements are in the same bucket;
  - Dissimilar elements are in different buckets.

Locality Sensitive Hashing: Big Picture



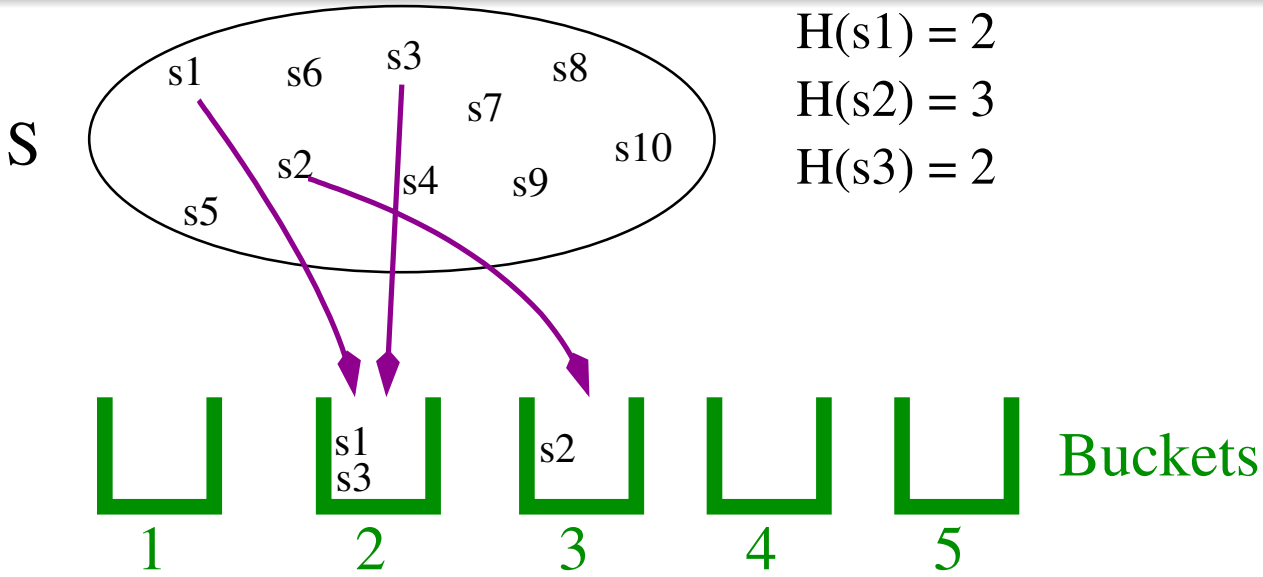
- Put elements of S into several "buckets" such that (with high probability):
  - Similar elements are in the same bucket;
  - Dissimilar elements are in different buckets.
  - The bucket of s is $H(s)$.

Locality Sensitive Hashing: Big Picture



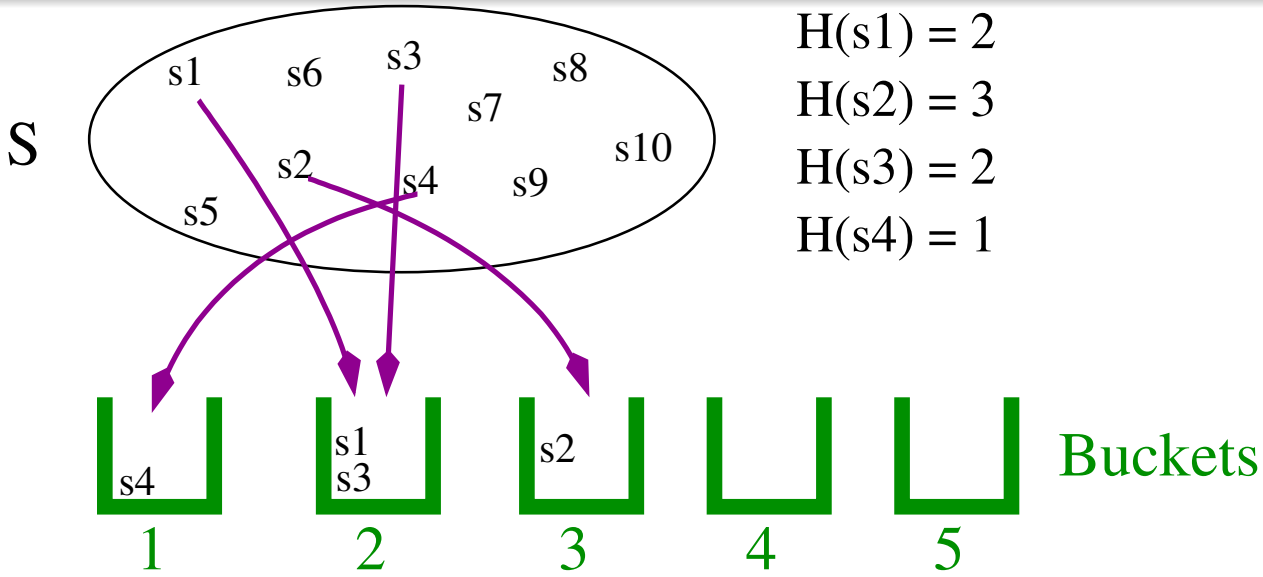
- Put elements of S into several "buckets" such that (with high probability):
  - Similar elements are in the same bucket;
  - Dissimilar elements are in different buckets.
  - The bucket of s is $H(s)$.

Locality Sensitive Hashing: Big Picture



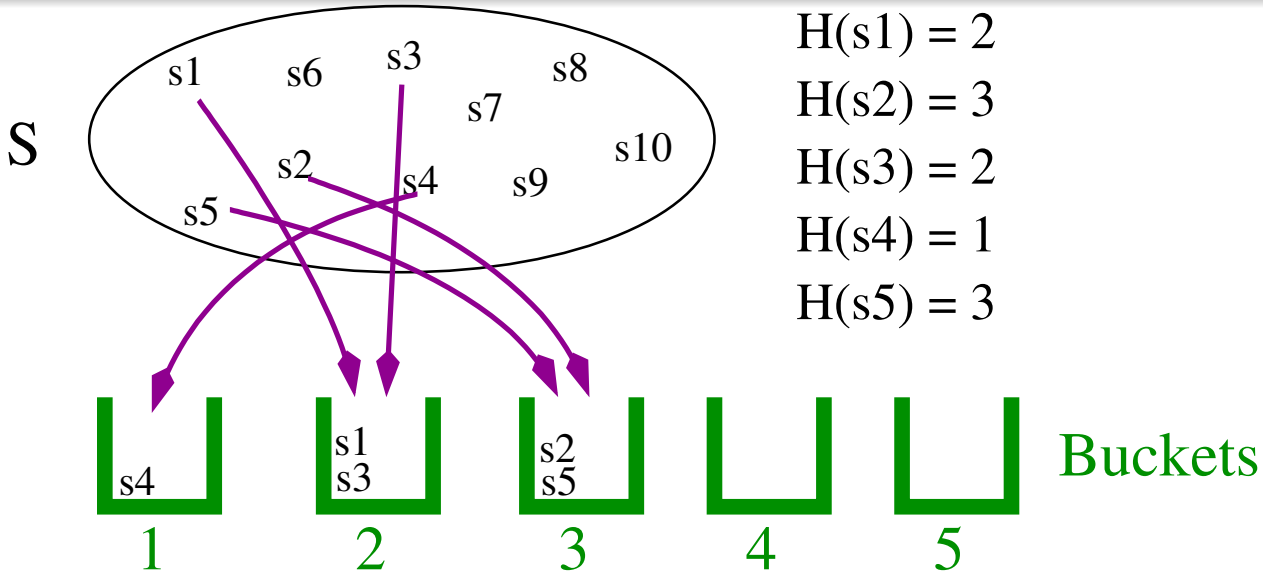
- Put elements of S into several "buckets" such that (with high probability):
  - Similar elements are in the same bucket;
  - Dissimilar elements are in different buckets.
  - The bucket of s is $H(s)$.

Locality Sensitive Hashing: Big Picture



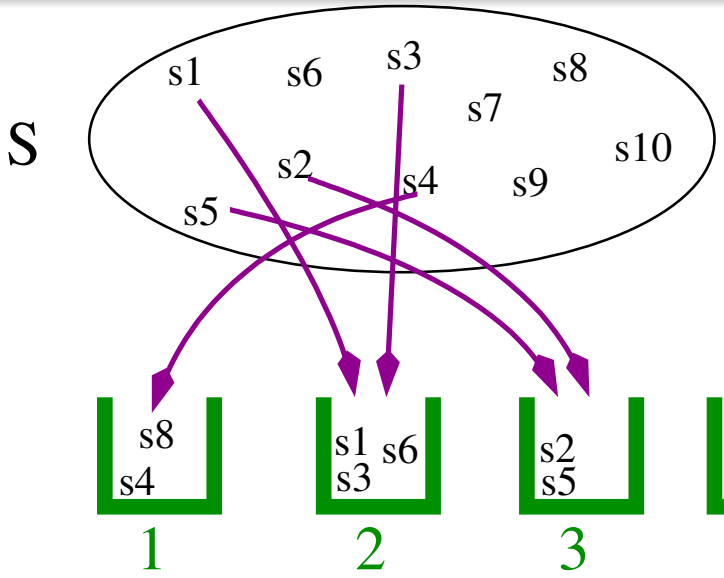
- Put elements of S into several "buckets" such that (with high probability):
  - Similar elements are in the same bucket;
  - Dissimilar elements are in different buckets.
  - The bucket of s is $H(s)$.

Locality Sensitive Hashing: Big Picture



- Put elements of S into several "buckets" such that (with high probability):
  - Similar elements are in the same bucket;
  - Dissimilar elements are in different buckets.
  - The bucket of s is $H(s)$.

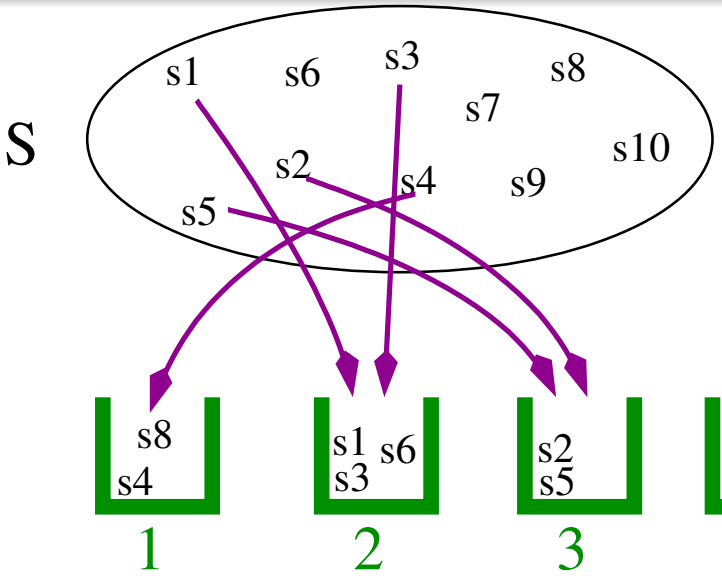
Locality Sensitive Hashing: Big Picture



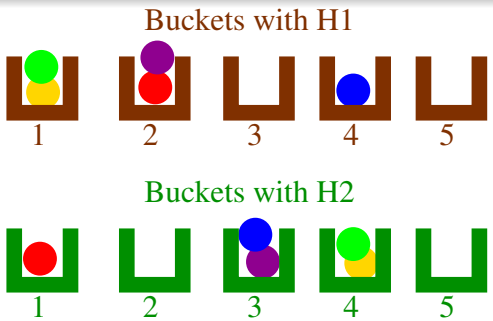
H(s1) = 2 H(s6) = 2
H(s2) = 3 H(s7) = 4
H(s3) = 2 H(s8) = 1
H(s4) = 1 H(s9) = 5
H(s5) = 3 H(s10) = 4

- Put elements of S into several "buckets" such that (with high probability):
  - Similar elements are in the same bucket;
  - Dissimilar elements are in different buckets.
  - The bucket of s is $H(s)$.

Locality Sensitive Hashing: Big Picture

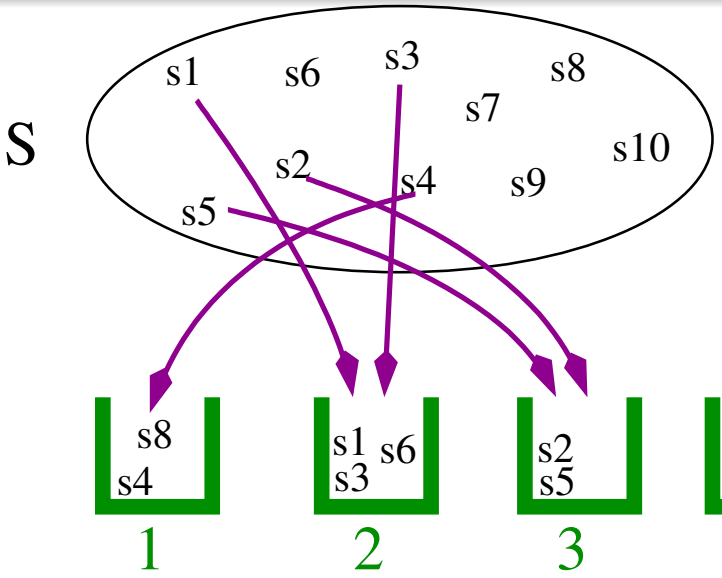


H(s1) = 2
H(s2) = 3
H(s3) = 2
H(s4) = 1
H(s5) = 3
H(s6) = 2
H(s7) = 4
H(s8) = 1
H(s9) = 5
H(s10) = 4

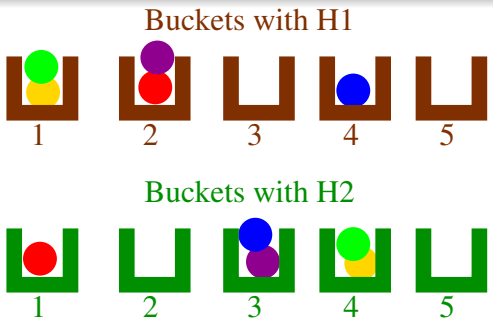


- Put elements of S into several "buckets" such that (with high probability):
  - Similar elements are in the same bucket;
  - Dissimilar elements are in different buckets.
  - The bucket of s is $H(s)$.
    - Can use several functions H_1, H_2, ...
    - $\Rightarrow$ several buckets for s: $H_1(s)$, $H_2(s)$, ...
    - The vector $(H_1(s), H_2(s)...)$ is the signature of s

Locality Sensitive Hashing: Big Picture



H(s1) = 2
H(s2) = 3
H(s3) = 2
H(s4) = 1
H(s5) = 3
H(s6) = 2
H(s7) = 4
H(s8) = 1
H(s9) = 5
H(s10) = 4

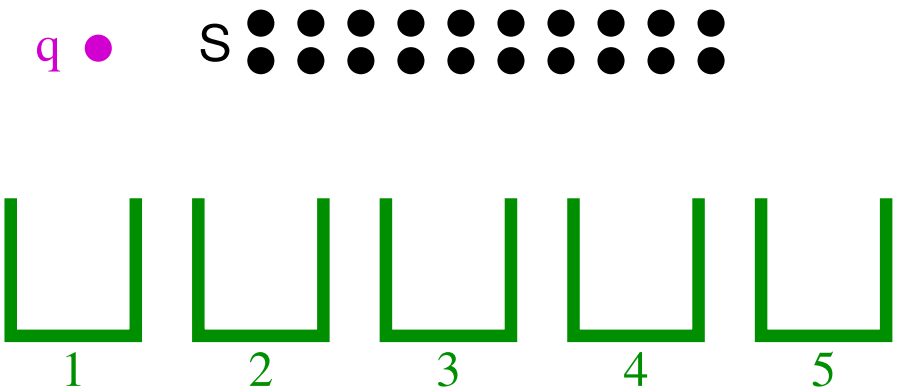


- Put elements of S into several "buckets" such that (with high probability):
  - Similar elements are in the same bucket;
  - Dissimilar elements are in different buckets.
  - The bucket of s is $H(s)$.
    - Can use several functions H_1, H_2, ...
    - $\Rightarrow$ several buckets for s: $H_1(s)$, $H_2(s)$, ...
    - The vector $(H_1(s), H_2(s)...)$ is the signature of s
- Objective: finding the signature (i.e., H_1, H_2, ...)

Using LSH for Nearest Neighbor

Find the most similar element in S to a query q:

- Compute $H(s)$ for every $s \in S$
- Compute $H(q)$
- Compute $\text{Sim}(x, q)$ for all x in the bucket $H(q)$
- Return the most similar



Complexity:

- If size of buckets is b: $O(n + b)$ (without LSH: $O(n)$)

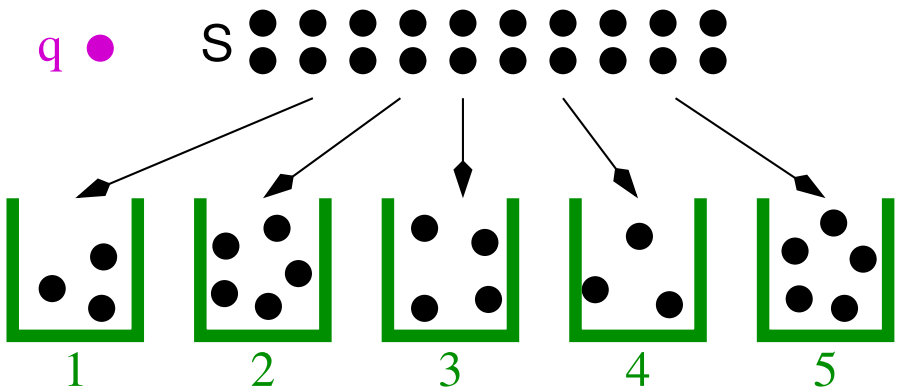
$\Rightarrow$ not interesting if only one query.

- with k queries q_1, ..., q_k:
  - $O(n + kb)$ (without LSH: $O(kn)$)

Using LSH for Nearest Neighbor

Find the most similar element in S to a query q:

- Compute $H(s)$ for every $s \in S$
- Compute $H(q)$
- Compute $\mathrm{Sim}(x, q)$ for all x in the bucket $H(q)$
- Return the most similar



Complexity:

- If size of buckets is b: $O(n + b)$ (without LSH: $O(n)$)

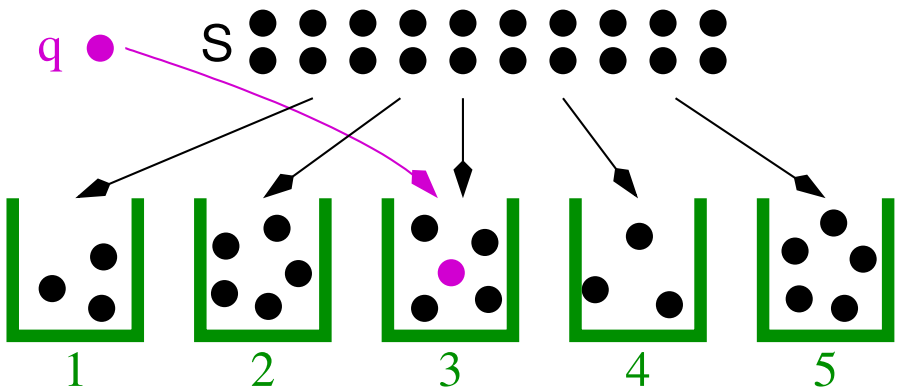
$\Rightarrow$ not interesting if only one query.

- with k queries q_1, ..., q_k:
  - $O(n + kb)$ (without LSH: $O(kn)$)

Using LSH for Nearest Neighbor

Find the most similar element in S to a query q:

- Compute $H(s)$ for every $s \in S$
- Compute $H(q)$
- Compute $\text{Sim}(x, q)$ for all x in the bucket $H(q)$
- Return the most similar



Complexity:

- If size of buckets is b: $O(n + b)$ (without LSH: $O(n)$)

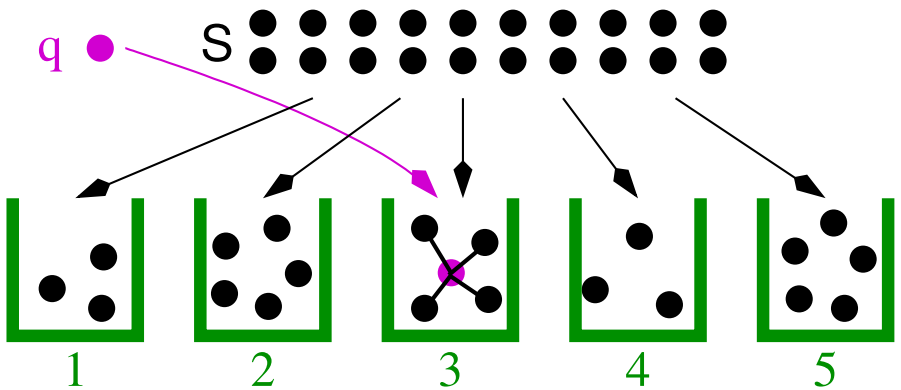
$\Rightarrow$ not interesting if only one query.

- with k queries q_1, ..., q_k:
  - $O(n + kb)$ (without LSH: $O(kn)$)

Using LSH for Nearest Neighbor

Find the most similar element in S to a query q:

- Compute $H(s)$ for every $s \in S$
- Compute $H(q)$
- Compute $\mathrm{Sim}(x, q)$ for all x in the bucket $H(q)$
- Return the most similar



Complexity:

- If size of buckets is b: $O(n + b)$ (without LSH: $O(n)$)

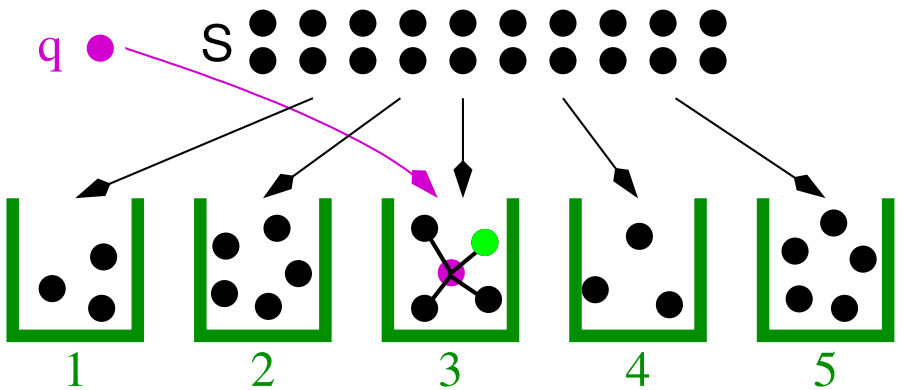
$\Rightarrow$ not interesting if only one query.

- with k queries q_1, ..., q_k:
  - $O(n + kb)$ (without LSH: $O(kn)$)

Using LSH for Nearest Neighbor

Find the most similar element in S to a query q:

- Compute $H(s)$ for every $s \in S$
- Compute $H(q)$
- Compute $\text{Sim}(x, q)$ for all x in the bucket $H(q)$
- Return the most similar



Complexity:

- If size of buckets is b: $O(n + b)$ (without LSH: $O(n)$)

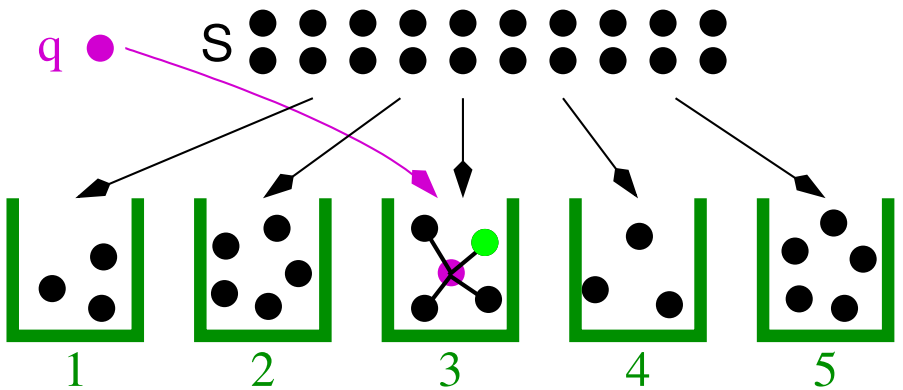
$\Rightarrow$ not interesting if only one query.

- with k queries q_1, ..., q_k:
  - $O(n + kb)$ (without LSH: $O(kn)$)

Using LSH for Nearest Neighbor

Find the most similar element in S to a query q:

- Compute $H(s)$ for every $s \in S$: $O(n)$
- Compute $H(q)$
- Compute $\mathrm{Sim}(x, q)$ for all x in the bucket $H(q)$: $O(b)$
- Return the most similar



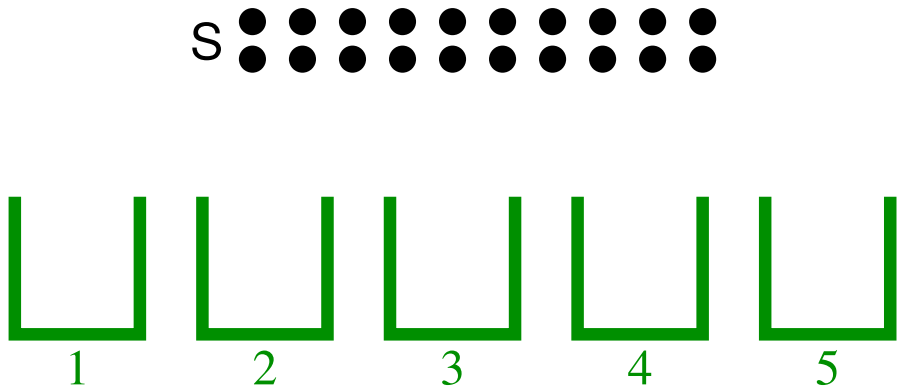
Complexity:

- If size of buckets is b: $O(n + b)$ (without LSH: $O(n)$)

$\Rightarrow$ not interesting if only one query.

- with k queries q_1, ..., q_k:
  - $O(n + kb)$ (without LSH: $O(kn)$)

Using LSH for Finding Groups of Similar Elements

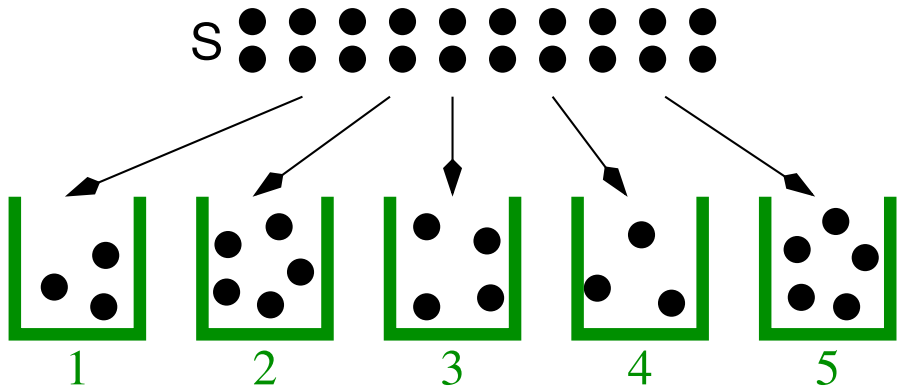


- Compute the buckets $H(s)$ for every $s \in S$
- Then compute similarity for pairs of elements in same bucket
  - compute $\mathrm{Sim}(x, y)$ for all x, y in same bucket.

Complexity:

- $O(n)$ to compute buckets
- For a bucket with b elements, $b(b-1)/2$ similarity computations
  - $O(b^2)$ for one bucket
- n/b buckets $\Rightarrow O(b^2 n/b) = O(bn)$ for all buckets
  (without LSH: $O(n^2)$)

Using LSH for Finding Groups of Similar Elements

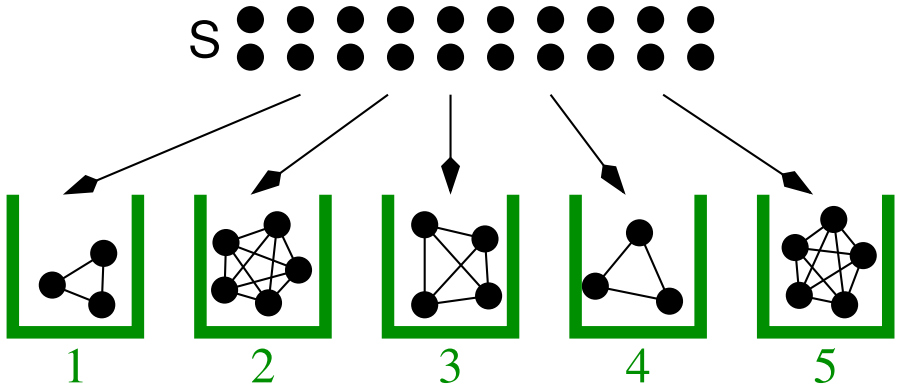


- Compute the buckets $H(s)$ for every $s \in S$
- Then compute similarity for pairs of elements in same bucket
  - compute $\mathrm{Sim}(x, y)$ for all x, y in same bucket.

Complexity:

- $O(n)$ to compute buckets
- For a bucket with b elements, $b(b-1)/2$ similarity computations
  - $O(b^2)$ for one bucket
- n/b buckets $\Rightarrow O(b^2 n/b) = O(bn)$ for all buckets
  (without LSH: $O(n^2)$)

Using LSH for Finding Groups of Similar Elements

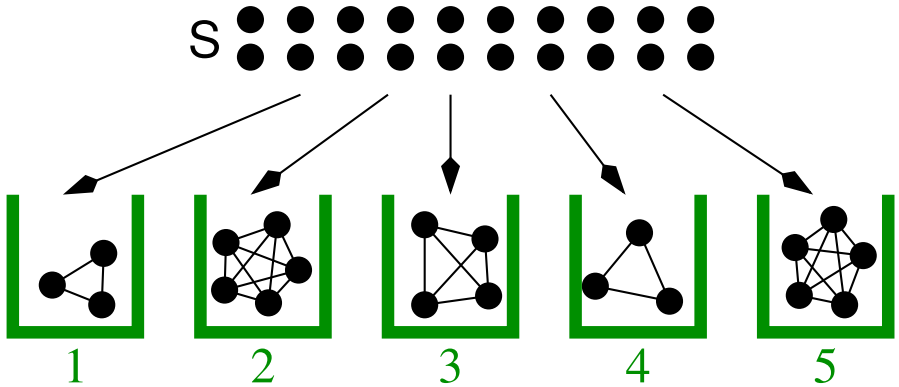


- Compute the buckets $H(s)$ for every $s \in S$
- Then compute similarity for pairs of elements in same bucket
  - compute $\text{Sim}(x, y)$ for all x, y in same bucket.

Complexity:

- $O(n)$ to compute buckets
- For a bucket with b elements, $b(b-1)/2$ similarity computations
  - $O(b^2)$ for one bucket
- n/b buckets $\Rightarrow O(b^2 n/b) = O(bn)$ for all buckets
  (without LSH: $O(n^2)$)

Using LSH for Finding Groups of Similar Elements



- Compute the buckets $H(s)$ for every $s \in S$
- Then compute similarity for pairs of elements in same bucket
  - compute $\text{Sim}(x, y)$ for all x, y in same bucket.

Complexity:

- $O(n)$ to compute buckets
- For a bucket with b elements, $b(b-1)/2$ similarity computations
  - $O(b^2)$ for one bucket
- n/b buckets $\Rightarrow O(b^2 n/b) = O(bn)$ for all buckets
  (without LSH: $O(n^2)$)

Part II: Application of LSH to Documents: Min Hashing

Application of LSH to Documents: Min Hashing

Given a set S of documents:

- Find similar documents
- Applications:
  - Document clustering, classification
  - Plagiarism detection
  - Fake news detection

Objective of this section: How to use LSH to find similar documents

- Need to define:
  - the similarity between documents: $\text{Sim}(D, D')$
  - the signature of a document D: $(H_1(D), H_2(D), ...)$

Similarity between documents

How to define similarity between two documents D_1 and D_2 ?

- Use Jaccard similarity (defined on sets)
- On sets of Shingles extracted in the documents.

Jaccard Similarity of Sets

Similarity defined for sets:

- $\text{Sim}_{\text{jac}}(A, B) = \frac{|A \cap B|}{|A \cup B|}$
- Example:
  - $A = \{a, b, c, d\}$; $B = \{c, d, e\}$
  - $\text{Sim}_{\text{jac}}(A, B) =$

Jaccard Similarity of Sets

Similarity defined for sets:

- $\text{Sim}_{\text{jac}}(A, B) = \frac{|A \cap B|}{|A \cup B|}$
- Example:
  - $A = \{a, b, c, d\}$; $B = \{c, d, e\}$
  - $\text{Sim}_{\text{jac}}(A, B) = |\{c, d\}| / |\{a, b, c, d, e\}| = 2/5$
- $0 \leq \text{Sim}_{\text{jac}}(A, B) \leq 1$
- Jaccard similarity between bit vectors:
  - Each bit is an element: 1 present; 0: absent
  - Ex: $\text{Sim}_{\text{jac}}((0, 0, 1, 1), (1, 0, 1, 1)) = 2/3$
- Document: set of words $\Rightarrow$ could use Jaccard similarity
  - But lose order of words

Shingling

Objective: representing a document by a set;

Keeping information on word order

- Optional: remove punctuation, stop words, capitalization, ...
- Extract k-shingles (k-grams):
  - Sequence of k consecutive characters / words
  - Ex: document = "St-Etienne is a nice city"
  - Character 3-shingles = {St-, t-E, -Et, Eti, tie, ien, enn, nne, ne_, e_i,_is,... ,ity}
  - Word 2-shingles = {St-Etienne, Etienne_is, is_a, a_nice, nice_city}
- Document Representation:
  - set of shingles (one shingle can appear at most once)
  - or bag / multiset of shingles (a shingle can appear several times)
- Optional: each shingle represented by a 32 bit integer (using a hash function)

Shingling

Objective: representing a document by a set;

Keeping information on word order

- Optional: remove punctuation, stop words, capitalization, ...
- Extract k-shingles (k-grams):
  - Sequence of k consecutive characters / words
  - Ex: document = "St-Etienne is a nice city"
  - Character 3-shingles = {St-, t-E, -Et, Eti, tie, ien, enn, nne, ne_, e_i,_is,... ,ity}
  - Word 2-shingles = {St-Etienne, Etienne_is, is_a, a_nice, nice_city}
- Document Representation:
  - set of shingles (one shingle can appear at most once)
  - or bag / multiset of shingles (a shingle can appear several times)
- Optional: each shingle represented by a 32 bit integer (using a hash function)

Shingling

Objective: representing a document by a set;

Keeping information on word order

- Optional: remove punctuation, stop words, capitalization, ...
- Extract k-shingles (k-grams):
  - Sequence of k consecutive characters / words
  - Ex: document = "St-Etienne is a nice city"
  - Character 3-shingles = {St-, t-E, -Et, Eti, tie, ien, enn, nne, ne_, e_i,_is,... ,ity}
  - Word 2-shingles = {St-Etienne, Etienne_is, is_a, a_nice, nice_city}
- Document Representation:
  - set of shingles (one shingle can appear at most once)
  - or bag / multiset of shingles (a shingle can appear several times)
- Optional: each shingle represented by a 32 bit integer (using a hash function)

Shingling

Objective: representing a document by a set;

Keeping information on word order

- Optional: remove punctuation, stop words, capitalization, ...
- Extract k-shingles (k-grams):
  - Sequence of k consecutive characters / words
  - Ex: document = "St-Etienne is a nice city"
  - Character 3-shingles = {St-, t-E, -Et, Eti, tie, ien, enn, nne, ne_, e_i,_is,... ,ity}
  - Word 2-shingles = {St-Etienne, Etienne_is, is_a, a_nice, nice_city}
- Document Representation:
  - set of shingles (one shingle can appear at most once)
  - or bag / multiset of shingles (a shingle can appear several times)
- Optional: each shingle represented by a 32 bit integer (using a hash function)

Shingling

Objective: representing a document by a set;

Keeping information on word order

- Optional: remove punctuation, stop words, capitalization, ...
- Extract k-shingles (k-grams):
  - Sequence of k consecutive characters / words
  - Ex: document = "St-Etienne is a nice city"
  - Character 3-shingles = {St-, t-E, -Et, Eti, tie, ien, enn, nne, ne_, e_i,_is,... ,ity}
  - Word 2-shingles = {St-Etienne, Etienne_is, is_a, a_nice, nice_city}
- Document Representation:
  - set of shingles (one shingle can appear at most once)
  - or bag / multiset of shingles (a shingle can appear several times)
- Optional: each shingle represented by a 32 bit integer (using a hash function)

Bit Vector Representation of Documents

- D_1 = "banana", D_2 = "ananas", D_3 = "anas", ...
- 3-shingles(D_1) =

Bit Vector Representation of Documents

- D_1 = "banana", D_2 = "ananas", D_3 = "anas", ...
- 3-shingles(D_1) = {ban, ana, nan}
- 3-shingles(D_2) =

Bit Vector Representation of Documents

- D_1 = "banana", D_2 = "ananas", D_3 = "anas", ...
- 3-shingles(D_1) = {ban, ana, nan}
- 3-shingles(D_2) = {ana, nan, nas}
- 3-shingles(D_3) =

Bit Vector Representation of Documents

- D_1 = "banana", D_2 = "ananas", D_3 = "anas", ...
- 3-shingles(D_1) = {ban, ana, nan}
- 3-shingles(D_2) = {ana, nan, nas}
- 3-shingles(D_3) = {ana, nas}
- Assign integers to all shingles (e.g., using hash function):

ban	ana	nan	nas	...
0	1	2	3	...

Bit Vector Representations:

- D_1 : (1, 1, 1, 0, ...)
- D_2 : (0, 1, 1, 1, ...)
- D_3 : (0, 1, 0, 1, ...)

Boolean Matrix Representation:

D_1	D_2	D_3	...
1	0	0	...
1	1	1	...
1	1	0	...
0	1	1	...
...			

Bit Vector Representation of Documents

- D_1 = "banana", D_2 = "ananas", D_3 = "anas", ...
- 3-shingles(D_1) = {ban, ana, nan}
- 3-shingles(D_2) = {ana, nan, nas}
- 3-shingles(D_3) = {ana, nas}
- Assign integers to all shingles (e.g., using hash function):

ban	ana	nan	nas	...
0	1	2	3	...

Bit Vector Representations:

- D_1 : (1, 1, 1, 0, ...)
- D_2 : (0, 1, 1, 1, ...)
- D_3 : (0, 1, 0, 1, ...)

Boolean Matrix Representation:

D_1	D_2	D_3	...
1	0	0	...
1	1	1	...
1	1	0	...
0	1	1	...
...			

Bit Vector Representation of Documents

- D_1 = "banana", D_2 = "ananas", D_3 = "anas", ...
- 3-shingles(D_1) = {ban, ana, nan}
- 3-shingles(D_2) = {ana, nan, nas}
- 3-shingles(D_3) = {ana, nas}
- Assign integers to all shingles (e.g., using hash function):

ban	ana	nan	nas	...
0	1	2	3	...

Bit Vector Representations:

- D_1 : (1, 1, 1, 0, ...)
- D_2 : (0, 1, 1, 1, ...)
- D_3 : (0, 1, 0, 1, ...)

Boolean Matrix Representation:

D_1	D_2	D_3	...
1	0	0	...
1	1	1	...
1	1	0	...
0	1	1	...
...			

Bit Vector Representation of Documents

- D_1 = "banana", D_2 = "ananas", D_3 = "anas", ...
- 3-shingles(D_1) = {ban, ana, nan}
- 3-shingles(D_2) = {ana, nan, nas}
- 3-shingles(D_3) = {ana, nas}
- Assign integers to all shingles (e.g., using hash function):

ban	ana	nan	nas	...
0	1	2	3	...

Bit Vector Representations:

- D_1 : (1, 1, 1, 0, ...)
- D_2 : (0, 1, 1, 1, ...)
- D_3 : (0, 1, 0, 1, ...)

Boolean Matrix Representation:

D_1	D_2	D_3	...
1	0	0	...
1	1	1	...
1	1	0	...
0	1	1	...
...			

Jaccard Similarity for Documents

Jaccard Similarity for Documents

- Each Document represented by a set of shingles
- Similarity between documents: Jaccard Similarity between the sets of shingles
- $\text{Sim}(D_1, D_2) = \text{Sim}_{\text{jac}}(\text{shingles}(D_1), \text{shingles}(D_2))$

Example (with sets of 3-shingles):

- D_1 = "banana"; $A = \text{shingle}(D_1) = \{\text{ban, ana, nan}\}$
- D_2 = "ananas"; $B = \text{shingle}(D_2) = \{\text{ana, nan, nas}\}$
- $\text{Sim}(D_1, D_2) = \text{Sim}_{\text{jac}}(A, B) = 2/4 = 0.5$

Min Hashing: Definition

Goal: find a signature (small vector) for sets that preserves similarity.

- Signature: combination of several functions min-hash$_p$.
- min-hash$_p(X)$ index of the first "1" value in X:
  - Document/set represented by a bit vector $X = (x_1, ...x_k)$
  - Given a permutation p
  - min-hash$_p(X) = min\{p(i) \mid x_{p(i)} = 1\}$
- Simple Algorithm to compute min-hash$_p$:
  - Sort the x_i according to $p(i)$
  - Find the first non zero value
  - min-hash$_p$: index of this value

Min Hashing Example

p_1	p_2	p_3	row	D_1	D_2	D_3	D_4
5	6	5	0	1	0	1	0
1	1	4	1	1	0	1	0
6	0	1	2	1	0	0	0
3	2	6	3	1	0	0	1
4	3	2	4	0	1	0	1
2	5	3	5	0	1	1	1
0	4	0	6	0	1	1	1

Min Hashing and Jaccard similarity

Probability two sets X and Y have the same min-hash$_p$:
Jaccard similarity $\mathrm{Sim}_{\mathrm{jac}}(X, Y)$!

Min Hashing and Jaccard similarity

Probability two sets X and Y have the same min-hash$_p$:
Jaccard similarity $\text{Sim}_{\text{jac}}(X, Y)$!

Why ?

- Given two sets represented by bit-vectors X and Y
- Three types of rows:
  - A 11;
  - B 01 or 10;
  - C 00
- a, b, c: number of A / B / C rows
- Jaccard similarity is

Min Hashing and Jaccard similarity

Probability two sets X and Y have the same min-hash$_p$:
Jaccard similarity $\mathrm{Sim}_{\mathrm{jac}}(X, Y)$!

Why ?

- Given two sets represented by bit-vectors X and Y
- Three types of rows:
  - A 11;
  - B 01 or 10;
  - C 00
- a, b, c: number of A / B / C rows
- Jaccard similarity is $\mathrm{Sim}_{\mathrm{jac}}(X, Y) = a/(a+b)$

Min Hashing and Jaccard similarity

Probability two sets X and Y have the same min-hash$_p$:
Jaccard similarity $\text{Sim}_{\text{jac}}(X, Y)$!

Why ?

- Given two sets represented by bit-vectors X and Y
- Three types of rows:
  - A 11;
  - B 01 or 10;
  - C 00
- a, b, c: number of A / B / C rows
- Jaccard similarity is $\text{Sim}_{\text{jac}}(X, Y) = a/(a+b)$
- Probability that min-hash$_p(X)$ = min-hash$_p(Y)$:
  - Remove C type rows

Min Hashing and Jaccard similarity

Probability two sets X and Y have the same min-hash$_p$:
Jaccard similarity $\text{Sim}_{\text{jac}}(X, Y)$!

Why ?

- Given two sets represented by bit-vectors X and Y
- Three types of rows:
  - A 11;
  - B 01 or 10;
  - C 00
- a, b, c: number of A / B / C rows
- Jaccard similarity is $\text{Sim}_{\text{jac}}(X, Y) = a/(a+b)$
- Probability that min-hash$_p(X)$ = min-hash$_p(Y)$:
  - Remove C type rows
  - First row is of type A iff min-hash$_p(X)$ = min-hash$_p(Y)$.

Min Hashing and Jaccard similarity

Probability two sets X and Y have the same min-hash$_p$:
Jaccard similarity $\text{Sim}_{\text{jac}}(X, Y)$!

Why ?

- Given two sets represented by bit-vectors X and Y
- Three types of rows:
  - A 11;
  - B 01 or 10;
  - C 00
- a, b, c: number of A / B / C rows
- Jaccard similarity is $\text{Sim}_{\text{jac}}(X, Y) = a/(a+b)$
- Probability that $\text{min-hash}_p(X) = \text{min-hash}_p(Y)$:
  - Remove C type rows
  - First row is of type A iff $\text{min-hash}_p(X) = \text{min-hash}_p(Y)$.
  - Probability (first row of type A) $= a/(a+b)$

Min Hashing Signature Representation

- Use several permutations (e.g., 100) p_1, p_2,...,p_{100}
- Signature of the document D: the vector of the min hash values. (min-hash$_{p1}(D)$, min-hash$_{p2}(D)$, min-hash$_{p3}(D)$,...)
- Signature matrix.

Problems:

1. Input matrix can be very large, e.g., 1 billion rows (the number of k-shingles)
2. Computing the permutation and accessing the rows in permuted order: very expensive
3. And there are many vectors X and permutations p
4. Simplification: instead of permutations: hash functions (almost a permutation)

Min Hashing Optimized Computation

- Row by row instead of column by column:
- Several vectors/documents D_1, D_2, $D_j = (D_{1,j}, ...D_{k,j})$, ... D_n.
  1. for all j, k initialize $minhash[k, j] = \infty$
  2. For each row i
  3. For each hash function h_k
  4. $h = h_k(i)$
  5. for each document D_j
  6. if $D_{i,j} = 1$ then
  7. $minhash[k, j] = min(minhash[k, j], h)$

row	D_1	D_2	D_3	D_4	h_1	h_2
0	1	0	1	0	1	2
1	1	0	0	0	0	3
2	0	1	0	1	3	0
3	0	1	1	1	2	1

- $h_1(x) = (3x + 1) \mod 4$
- $h_2(x) = (x + 2) \mod 4$

Min Hashing for Locality Sensitive Hashing

Goal: given a large set S of documents, find pairs of similar documents.

- Compute documents bit-vector representation
  - binary vector representation of set of k-shingles
- Compute documents signatures
  - Given a set of hash functions h_1, h_2, ..., h_k
  - Signature of D is $(minhash_{h1}(D), ..., minhash_{hk}(D))$
- Each signature divided into b bands of r rows
- If two signatures are equal for each row of a band:
  - Similar with high probability: candidate pairs
- Otherwise (different in every band)
  - Dissimilar with high probability

D_1	D_2	D_3	D_4
3	2	3	2
1	1	1	2
4	3	4	4
2	1	1	2
1	4	4	4
1	3	1	3
2	1	1	1
3	1	3	1

4 bands of 2 rows.

Min Hashing for Locality Sensitive Hashing

- Candidate pairs: Documents similar with high probability
- Check if actually similar:
- $\Rightarrow$ Compute Jaccard similarity for candidate pairs:
  - $\text{Sim}_{\text{jac}}(D_1, D_3)$
  - $\text{Sim}_{\text{jac}}(D_1, D_4)$
  - $\text{Sim}_{\text{jac}}(D_2, D_4)$

D_1	D_2	D_3	D_4
3	2	3	2
1	1	1	2
4	3	4	4
2	1	1	2
1	4	4	4
1	3	1	3
2	1	1	1
3	1	3	1

4 bands of 2 rows.

Next Course

- What is this probability ?
- How to choose number of bands / number of rows ?
- LSH Amplification
- LSH for vectors (instead of documents)